MILESTONE 1 REPORT

Predicting Gaming Addiction: Linear Regression, Logistic Regression and KNN

392443 – 400305 – 399383

April 24, 2026

1. Introduction

The dataset contains 2000 samples and 13 numerical features describing gaming habits, split into 1600 training samples and 400 test samples. We tackle two tasks in parallel: *(i)* regression of the addiction score (integer in $[0, 10]$) and *(ii)* classification of the addiction level into three classes {Low, Medium, High}. To select hyperparameters, we reserve 20% of the training set as a **validation set**: the model is fitted on the remaining 80% and evaluated on the validation set for each candidate hyperparameter. The held-out test set is used *only* for the final evaluation reported in Table 1, so as not to bias our estimate of generalization.

2. Methods

2.1. Data preprocessing

All features are standardized with a **z-score**: $x_{\text{norm}} = (x - \mu)/\sigma$, where μ and σ are estimated on the training set only (to avoid data leakage) and applied to both sets. A **bias column** is appended for the two linear models; KNN uses no bias.

2.2. Linear Regression

We use the closed-form normal equation,

$$\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y},$$

which minimizes the MSE exactly. No hyperparameter is required, which makes it a natural baseline.

2.3. Logistic Regression

For $C = 3$ classes, we learn a weight matrix $W \in \mathbb{R}^{D \times C}$ by gradient descent on the softmax cross-entropy:

$$\nabla_W \mathcal{E}(W) = X^\top \big(\text{softmax}(XW) - T\big),$$

with T the one-hot label matrix. Weights are initialized as $\mathcal{N}(0, 0.1^2)$ and updated as $W \leftarrow W - \eta \nabla_W \mathcal{E}(W)$. Hyperparameters: learning rate η and number of iterations.

2.4. K-Nearest Neighbors

KNN is non-parametric: for each test point, we compute Euclidean distances to all training points, keep the K closest, and aggregate their labels. We use majority voting (`argmax` of `bincount`) for classification and the mean for regression. The same implementation serves both tasks via a `task_kind` flag.

3. Experiments and Results

3.1. Hyperparameter selection on the validation set

For each candidate hyperparameter, we train on the 1280 training samples, evaluate on the 320 validation samples, and plot **both train and validation** metrics. The gap between the two curves is a direct visual cue of over/underfitting.

KNN regression. We swept $K \in \{1, \dots, 75\}$ (Figure 1). At $K = 1$, train MSE is exactly 0 (the model memorizes every sample) while validation MSE reaches 4.07: this is **pure overfitting**. As K grows, train MSE rises while validation MSE drops, and both curves converge around $K \approx 10$–20. The validation minimum is at $K = 12$ with MSE $= 2.15$. Beyond $K \approx 25$, both curves rise together – the model is **underfitting**, averaging over too many irrelevant neighbors.

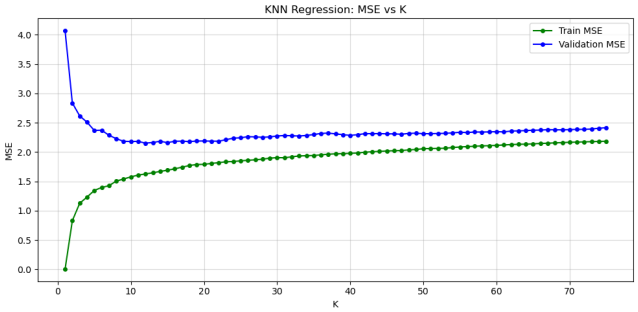


Figure 1: KNN regression: MSE on train (green) and validation (blue) vs. K. The large gap at $K = 1$ and its reduction for $K \geq 10$ illustrate the bias-variance trade-off. Best: $K = 12$, val MSE $= 2.15$.

KNN classification. We observe the same pattern (Figure 2). At $K = 1$, train accuracy is 100% and F1 is 1.0, while validation F1 is only 0.51: textbook overfitting. Validation accuracy remains relatively stable around 73–78% across the whole range, which is misleading on this imbalanced dataset. The macro F1-score is more informative: it peaks at $K = 15$ with F1 $= 0.574$ and degrades clearly for larger K, as minority classes get systematically outvoted.

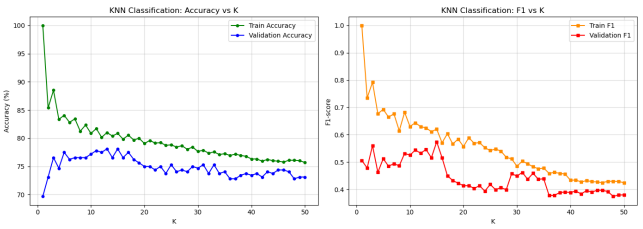


Figure 2: KNN classification: accuracy (left) and macro F1 (right) on train vs. validation. Best F1 at $K = 15$ (val F1 $= 0.574$).

Logistic regression. We performed a joint grid search over $\eta \in \{10^{-6}, 5\times10^{-6}, 10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}\}$ and `max_iters` $\in \{100, 250, 500, 750, 1000, 1500, 2000\}$ (Figure 3). Very small learning rates fail to converge within the allowed iterations (F1 stuck around 0.27–0.54). Increasing either η or the number of iterations moves the model towards a plateau. The training and validation surfaces remain close throughout, suggesting no major overfitting. The best validation configuration is $\eta = 10^{-3}$, `max_iters` $= 1500$, with F1 $= 0.760$ on validation.

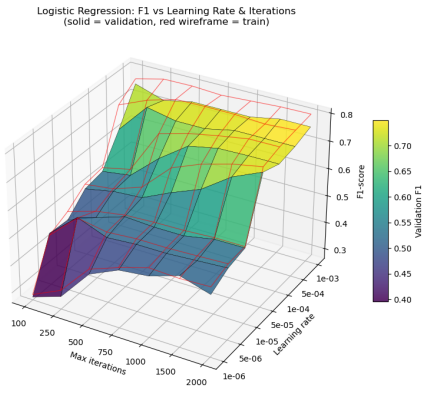


Figure 3: Logistic regression: validation F1 (solid surface) and training F1 (red wireframe) as a function of η and `max_iters`. High learning rates with enough iterations are both necessary.

3.2. Final performance on the test set

With the hyperparameters selected above, we re-train each model on the full 1600-sample training set and evaluate *once* on the 400-sample held-out test set (Table 1). All methods meet or exceed the project benchmarks: Linear Regression reaches MSE $= 0.99 < 1.1$; Logistic Regression achieves 88.75% accuracy $> 80\%$; KNN classification reaches 80.0% accuracy $> 70\%$; KNN regression attains MSE $= 1.86 < 2.1$.

Table 1: Final results on the held-out test set (400 samples), using hyperparameters chosen on the validation set.

Method	Task	Acc. / F1	MSE
Linear Regression	Regression	–	0.994
Logistic Regression ($\eta = 10^{-3}$)	Classification	88.75% / 0.863	–
KNN ($K = 15$)	Classification	80.00% / 0.519	–
KNN ($K = 12$)	Regression	–	1.858

4. Discussion and Conclusion

Overfitting vs. underfitting. The most striking observation across all three methods is the direct visualization of the bias-variance trade-off on the KNN curves (Figures 1 and 2). At $K = 1$, the train-validation gap is enormous (train F1 $= 1.0$ vs. val F1 $= 0.51$ for classification; train MSE $= 0$ vs. val MSE $= 4.07$ for regression) – a textbook case of overfitting in a memory-based model. As K grows, this gap closes, but both curves then start rising together for $K > 25$, which is the **underfitting** regime: averaging over too many distant neighbors smooths the predictions below what the true signal supports. The sweet spot ($K \approx 12$–15) balances these two extremes, exactly as predicted by the bias-variance theory.

Linear Regression achieves a test MSE of 0.99, below the 1.1 benchmark. The train (0.99) and test (0.99) MSEs are essentially identical, meaning the model generalizes almost perfectly – a sign that the relationship between features and addiction score is truly close to linear and that our 14-parameter model has no excess capacity to overfit.

Logistic Regression benefits substantially from proper tuning: a low learning rate (10^{-6}) with few iterations yields validation F1 ≈ 0.27, while $\eta = 10^{-3}$ with 1500 iterations reaches 0.76 on validation and 0.86 on the test set. The train/validation surfaces remain close throughout the grid, indicating that logistic regression does *not* overfit significantly here – the main failure mode is *under*-convergence rather than memorization. Interestingly, the test F1 (0.86) is even higher than the validation F1 (0.76), which simply reflects that the held-out test set happens to be slightly easier than the particular 320-sample validation fold we drew.

KNN is harder to tune than expected: while $K = 12$ is optimal for regression, classification peaks at $K = 15$, showing that the right complexity depends on both the task and the loss/metric considered. On classification, KNN's test F1 (0.519) remains well below that of logistic regression (0.863), suggesting the decision boundaries in the normalized 13-dimensional space are closer to linear than to the locally-constant regions KNN can capture.

Overall, the two linear models dominate KNN on both tasks. A key takeaway is that *data preprocessing* (z-score normalization) and *hyperparameter selection* contribute at least as much as the choice of model. Natural extensions would be L2-regularized (ridge) regression, k-fold cross-validation instead of a single validation split for more robust tuning, and basic feature engineering (interactions, polynomial expansion) to test whether the linearity assumption still holds.